Some Interview code Questions

1. Event Loop: Callbacks vs. Microtasks

JavaScript

```
console.log('1');

setTimeout(() => {
  console.log('2');
}, 0);

Promise.resolve().then(() => {
  console.log('3');
  setTimeout(() => console.log('4'), 0);
}).then(() => {
  console.log('5');
});

console.log('6');
```

Output:

Plaintext

```
1
6
3
5
2
4
```

Why: Synchronous code (`1`, `6`) executes first. Microtasks (`Promise.then` callbacks) run before macrotasks (`setTimeout`). Inside the microtask queue, `3` prints, enqueues another macrotask (`4`), and then the second `.then()` prints `5`. Finally, macrotasks execute in FIFO order (`2` then `4`).

2. Closures & Block Scope (`var` vs `let`)

JavaScript

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log('var:', i), 10);
}

for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log('let:', j), 10);
}
```

Output:

Plaintext

```
var: 3
var: 3
var: 3
let: 0
let: 1
```

```
let: 2
```

Why: `var` is function-scoped. All three iterations share a single `i` variable in memory, which reaches `3` by the time the timers fire. `let` is block-scoped, creating a fresh variable binding for every iteration that the arrow function closes over.

3. Scope & Temporal Dead Zone (TDZ)

JavaScript

```
var x = 10;

function checkTDZ() {
  console.log(x);
  let x = 20;
}

checkTDZ();
```

Output:

Plaintext

```
ReferenceError: Cannot access 'x' before initialization
```

Why: `let` variables are hoisted to the top of their enclosing block (`checkTDZ`), but they remain uninitialized in the **Temporal Dead Zone (TDZ)**. Accessing `x` inside the function shadows the global `var x = 10` and throws a `ReferenceError`.

4. `this` Context & Implicit Binding Loss

JavaScript

```
const user = {
  name: 'Alex',
  greet() {
    console.log(`Hello, ${this.name}`);
  },
  greetArrow: () => {
    console.log(`Hello, ${this.name}`);
  }
};

const unboundGreet = user.greet;

user.greet();
unboundGreet();
user.greetArrow();
```

Output:

Plaintext

```
Hello, Alex
Hello, undefined
Hello, undefined
```

Why:

1. `user.greet()` maintains `user` as its implicit `this` binding.
2. `unboundGreet()` detaches the function reference, causing `this` to default to `undefined` (in strict mode) or the global `window` object.
3. Arrow functions do not have their own `this`; they inherit `this` lexically from the enclosing outer scope (which is global/module level here, not the object literal).

5. Modern JS: Built-in Set Methods & `Promise.try`

JavaScript

```
const setA = new Set([1, 2, 3]);
const setB = new Set([3, 4, 5]);

// Built-in Set Union
const combined = setA.union(setB);
console.log([...combined]);

// Promise.try handles both synchronous and asynchronous errors uniformly

Promise.try(() => {
  throw new Error('Sync Failure');
})
.catch(err => console.log('Caught:', err.message));

console.log('Execution Finished');
```

Output:

Plaintext

```
[1, 2, 3, 4, 5]
Execution Finished
Caught: Sync Failure
```

Why: Built-in Set methods (`union`, `intersection`, `difference`) allow set operations directly without manual filtering. `Promise.try` wraps synchronous code in a Promise chain; because synchronous code outside the promise executes first, `'Execution Finished'` logs before the microtask `.catch()` callback runs.